Coding Fluency: Workflows & Best Practices

Suman 20 Jan, 2026 At 8:30 PM Pre-Reads Trimester-1 Recommended Resource

Associated Lectures (1)

Description Discussions

SESSION 1.3 PRE-READ

Control Flow, Conditionals & Loops

Program: AIM101 — Artificial Intelligence & Machine Learning Foundation **Pre-read Status:** MANDATORY — Must be completed before live session

Session 1.2 Recap: What You Already Know

Before Session 1.3, make sure you are comfortable with these concepts from Session 1.2:

Type Conversion

Converting between data types:

```
int("25")      # String to integer: 25
float("3.14")  # String to float: 3.14
str(42)        # Integer to string: "42"
bool(1)        # Integer to boolean: True
```

Arithmetic Operators

Operator	Name	Example	Result
`//`	Floor Division	`10 // 3`	`3`
`%`	Modulo (Remainder)	`10 % 3`	`1`
`**`	Exponentiation	`2 ** 3`	`8`

String Indexing and Slicing

Accessing characters and portions of strings:

```
text = "Python"
text[0]      # First character: "P"
text[-1]     # Last character: "n"
text[1:4]    # Slice from index 1 to 3: "yth"
text[:3]     # First 3 characters: "Pyt"
```

What You Will Learn Today

By the end of Session 1.3, you will be able to:

- **Apply** string methods like `.upper()` , `.lower()` , `.strip()` , `.replace()` , `.split()`
- **Format** output professionally using f-strings with alignment and decimal precision
- **Build** interactive programs using `print()` and `input()`
- **Compare** values using comparison operators (`==` , `!=` , `<` , `>` , `<=` , `>=`)
- **Combine** conditions using logical operators (`and` , `or` , `not`)
- **Make decisions** with `if` / `elif` / `else` statements
- **Create** nested conditionals for complex decision trees
- **Repeat** actions using `for` loops with `range()`
- **Implement** condition-based repetition with `while` loops
- **Control** loop execution with `break` and `continue`
- **Build** a complete Number Guessing Game combining all concepts

The Business Story: Why Automation Matters

The Power of Automation in Real Systems

Imagine you are working at a bank processing loan applications. Every day, hundreds of applications arrive. For each one, you must check:

- Is the applicant over 18?
- Is their credit score above 650?
- Is their income sufficient for the loan amount?
- Have they been employed for more than 2 years?

Without programming: A human reviews each application manually. With 500 applications per day, and 5 minutes per review, that is 41+ hours of work daily. Errors creep in from fatigue. Decisions are inconsistent.

With conditionals and loops:

```
for application in applications:
    if age >= 18 and credit_score > 650 and income_ratio > 0.3:
        approve(application)
```

```
    else:
        flag_for_review(application)
```

The same 500 applications are processed in seconds with perfect consistency.

Real-World Impact

Netflix Recommendations: Uses conditionals to decide what to show you based on your viewing history. "If user watched action movies, then recommend more action movies."

Spam Filters: Loop through every incoming email, checking conditions: "Does it contain suspicious words? Is the sender unknown? If yes, move to spam."

Traffic Lights: A simple loop with conditions controls traffic flow across the world, preventing collisions and optimizing traffic.

Video Games: Every game is a giant loop that keeps running until the player quits, constantly checking conditions: "Did the player press jump? Is the character touching the ground? Did they collide with an enemy?"

Today's Session: You will build a Number Guessing Game where the computer picks a secret number, and you try to guess it. The computer will tell you "too high" or "too low" until you win. This requires:

- **Conditionals** to check your guess
- **Loops** to let you keep guessing
- **All the concepts** from this session working together

Setup Requirements

Quick Environment Check (60 seconds)

Your environment should already be set up from Sessions 1.1 and 1.2. Just verify:

Open Terminal/Command Prompt

Activate your environment:

```
conda activate mlenv
```

Check Python version:

```
python --version
```

Expected: `Python 3.12.x`

Launch Jupyter:

```
jupyter notebook
```

Expected: Browser opens with Jupyter interface

If any step fails, refer back to the Session 1.1 Pre-read for troubleshooting.

Concepts to Preview

A. String Methods

Methods are functions that belong to objects. For strings, they transform text:

```
name = "  priya sharma  "

name.upper()      # "  PRIYA SHARMA  "
name.lower()      # "  priya sharma  "
name.strip()      # "priya sharma" (removes whitespace)
name.title()      # "  Priya Sharma  "
name.replace("a", "o")  # "  priyo shormo  "
```

Key insight: String methods return a NEW string. The original is unchanged.

B. F-Strings (Formatted String Literals)

The modern way to build strings with variables:

```
name = "Raj"
age = 22
score = 95.678

# Basic f-string
print(f"Hello, {name}!")          # Hello, Raj!

# Math inside f-string
print(f"Next year: {age + 1}")      # Next year: 23

# Formatting numbers
print(f"Score: {score:.2f}")       # Score: 95.68
print(f"Score: {score:.0f}")       # Score: 96
```

C. Input and Output

`print()` sends output to the screen. `input()` receives text from the user:

```
name = input("Enter your name: ")
print(f"Hello, {name}!")
```

Critical: `input()` ALWAYS returns a string! To get a number:

```
age = int(input("Enter your age: "))
height = float(input("Enter your height: "))
```

D. Comparison Operators

These return `True` or `False` :

Operator	Meaning	Example	Result
`==`	Equal to	`5 == 5`	`True`
`!=`	Not equal	`5 != 3`	`True`
`>`	Greater than	`5 > 3`	`True`
`<`	Less than	`5 < 3`	`False`
`>=`	Greater or equal	`5 >= 5`	`True`
`<=`	Less or equal	`5 <= 3`	`False`

Warning: Do not confuse `=` (assignment) with `==` (comparison)!

E. Logical Operators

Combine multiple conditions:

Operator	Meaning	Example
`and`	Both must be True	`age >= 18 and has_license`
`or`	At least one True	`is_weekend or is_holiday`
`not`	Flips True/False	`not is_raining`

F. Conditionals (if/elif/else)

Make decisions in your code:

```
temperature = 35

if temperature > 30:
    print("It's hot!")
elif temperature > 20:
    print("It's nice.")
else:
    print("It's cold.")
```

Key points:

- Colon `:` after each condition
- Indentation (4 spaces) defines the code block
- Only ONE branch executes

G. Loops

for loop — repeat a specific number of times:

```
for i in range(5):
    print(i)  # Prints 0, 1, 2, 3, 4
```

while loop — repeat until a condition is False:

```
count = 1
while count <= 5:
    print(count)
    count = count + 1  # Must change condition!
```

Key Vocabulary

Familiarize yourself with these terms before class:

Conditions and Logic

Term	Definition
Condition	An expression that evaluates to True or False
Boolean	A data type with only two values: True or False
Comparison	Testing the relationship between two values
Logical Operator	Combines or modifies boolean values (and, or, not)

Flow Control

Term	Definition
Conditional	Code that runs only when a condition is True
Branch	One path of execution in an if/elif/else structure
Nested	A structure (like an if) placed inside another structure

Loops

Term	Definition
Iteration	One pass through a loop; repeating a process
Loop	A code structure that repeats a block of code
Infinite Loop	A loop that never stops (usually a bug!)
break	A keyword that exits a loop immediately
continue	A keyword that skips to the next iteration
range()	A function that generates a sequence of numbers

Things to Ponder

Think about these questions before class. We will explore them during the session.

String and I/O Questions

Why does `input()` always return a string, even when the user types a number?

- What would happen if you try `age = input("Age: ")` and then `age + 1`?
- How would you fix this?

What is the difference between `.strip()` and `.replace(" ", "")`?

- When would you use each one?

Conditional Questions

Why do we need `elif` when we could use multiple `if` statements?

```
# Option A: if/elif/else
if grade >= 90:
    print("A")
elif grade >= 80:
    print("B")

# Option B: Multiple ifs
if grade >= 90:
    print("A")
if grade >= 80:
    print("B")
```

- What is different about how these execute?
- What happens if grade = 95?

What is the output of this code?

```
x = 5
if x > 3 and x < 10:
    print("In range")
```

- How does `and` work here?

Loop Questions

What does `range(1, 10, 2)` produce?

- What are the three arguments?
- How is the stop value treated?

When would you use `for` vs `while`?

- "Print numbers 1 to 10" — which loop?
- "Keep asking until user types 'quit'" — which loop?

What happens if you forget to update the condition in a while loop?

```
count = 1
while count < 5:
    print(count)
    # Forgot: count = count + 1
```

Preview: The Number Guessing Game

Here is what you will build by the end of class:

```
========================================
        NUMBER GUESSING GAME
========================================

Choose difficulty:
1. Easy (1-50)
2. Medium (1-100)
3. Hard (1-500)

Enter choice (1-3): 2

I'm thinking of a number between 1 and 100.
You have unlimited guesses. Good luck!

Your guess: 50
Too low! Try again.

Your guess: 75
Too high! Try again.

Your guess: 62
```

```
Too low! Try again.

Your guess: 68
Correct! You got it in 4 attempts!

========================================
        THANKS FOR PLAYING!
========================================
```

This program combines:

- **String methods & f-strings**: Creating the menu and messages
- **Input/Output**: Getting and displaying user interaction
- **Comparison operators**: Checking if guess is correct, too high, or too low
- **Logical operators**: Validating input is in range
- **if/elif/else**: Providing appropriate feedback
- **while loop**: Letting the player keep guessing until correct
- **break**: Exiting when they win

Optional Deep Dive

Python Official Documentation

- **String Methods**: https://docs.python.org/3.12/library/stdtypes.html#string-methods
- **Control Flow**: https://docs.python.org/3.12/tutorial/controlflow.html
- **range()**: https://docs.python.org/3.12/library/stdtypes.html#range

Interactive Practice

Try these in your Jupyter notebook before class:

```
# String methods
"  hello world  ".strip()
"hello".upper()
"Hello World".replace("World", "Python")
"a,b,c".split(",")

# Comparisons
5 > 3
5 == 5
5 != 5

# Logical operators
True and False
True or False
not True

# Simple loop
```

```python
for i in range(5):
    print(i)
```

Thinking Exercise

Consider this real-world scenario:

A vending machine needs to:

- Display available items
- Get user's selection
- Check if valid selection
- Check if user has enough money
- Dispense item or show error
- Ask if they want another item
- Keep running until user says "no"

Question: Which parts need conditionals? Which need loops?

Pre-Class Checklist

Before coming to class, ensure you have:

- [ ] Read this entire document
- [ ] Reviewed Session 1.2 concepts (type conversion, operators, string indexing)
- [ ] Verified your Python environment works (`conda activate mlenv`)
- [ ] Tried the practice code in Jupyter notebook
- [ ] Thought about the "Things to Ponder" questions
- [ ] Understood what the Number Guessing Game will do

See You in Class!

Today you will learn to make programs that THINK (conditionals) and REPEAT (loops). These are the building blocks of every intelligent application, from simple games to complex AI systems.

Get ready to build your first interactive game!

Vishlesan i-Hub IIT Patna × Masai School